

Mini Hiring Pipeline

Project Design & Architecture Document

GitHub Repository: <https://github.com/SAfsehEhsani/Mini-Hiring-Pipeline-Career-Passport->

1. Project Overview

Mini Hiring Pipeline is a recruiter-focused web application for managing candidates through a single hiring pipeline. Candidates move through Applied, Screening, Interview, Offer and Hired. A candidate can be rejected before being hired, and final outcomes cannot be reversed.

2. Main Features

- Add new candidates to the hiring pipeline.
- View candidates grouped by their current stage.
- Move candidates forward one stage at a time.
- Reject candidates from any stage before Hired.
- Prevent skipping stages or reversing final outcomes.
- Open a candidate and view their complete history.
- Show how long a candidate has been in the current stage.
- Maintain an append-only audit trail for candidate events.
- Search candidates using a single natural-language search box.
- Rank the best matching candidates first.
- Explain unsupported or unsuccessful searches instead of showing only an empty result.
- Persist the demo data locally in the browser.

3. Technology Used

The project is built as a client-side React application using React 19, TypeScript and Vite. Candidate data is persisted using browser localStorage. The search functionality is implemented locally using deterministic search and matching logic, so the application does not require a backend server or an external AI API to run.

4. Application Architecture

The application follows a simple frontend architecture. The React UI manages the recruiter experience, while separate utility modules handle the pipeline state machine, search logic, date and duration calculations, and audit history.

The main flow is:

React UI → Application State → Pipeline / Search Utilities → Browser localStorage

5. Hiring Pipeline

The normal hiring flow is:

Applied → Screening → Interview → Offer → Hired

Candidates can be rejected from Applied, Screening, Interview or Offer. Rejected and Hired are terminal states, so they cannot be moved to another outcome.

6. Stage Transition Rules

- A candidate can move only to the next valid stage.
- Skipping a stage is not allowed.
- Moving backwards is not allowed.
- A Hired candidate cannot be changed.
- A Rejected candidate cannot be changed.
- Every valid transition creates a new audit event.

7. Audit History

Every candidate has an audit trail. Candidate creation, stage transitions and rejection events are recorded with timestamps and relevant information. The history is used to show the complete journey of a candidate and to answer historical search questions.

The repository uses a client-side tamper-evidence signature for audit entries. This is suitable for demonstrating the concept in the take-home project; a production implementation would use server-side cryptographic protection such as HMAC/SHA-256.

8. Current Stage Duration

The application records when a candidate entered the current stage. The elapsed time is calculated from that timestamp and displayed to the recruiter. Candidates who have remained in a stage for seven or more days can be treated as stalled.

9. Search

The application provides one search box for recruiter questions. The search engine interprets supported natural language patterns and converts them into deterministic filters.

Examples

- “Find Priya Sharma” — finds the candidate by name.
- “sharam” — fuzzy matching can still find “Sharma”.
- “Who is in Interview right now?” — finds candidates currently in Interview.
- “Who has been stuck in Screening for more than a week?” — finds Screening candidates who have exceeded the duration threshold.

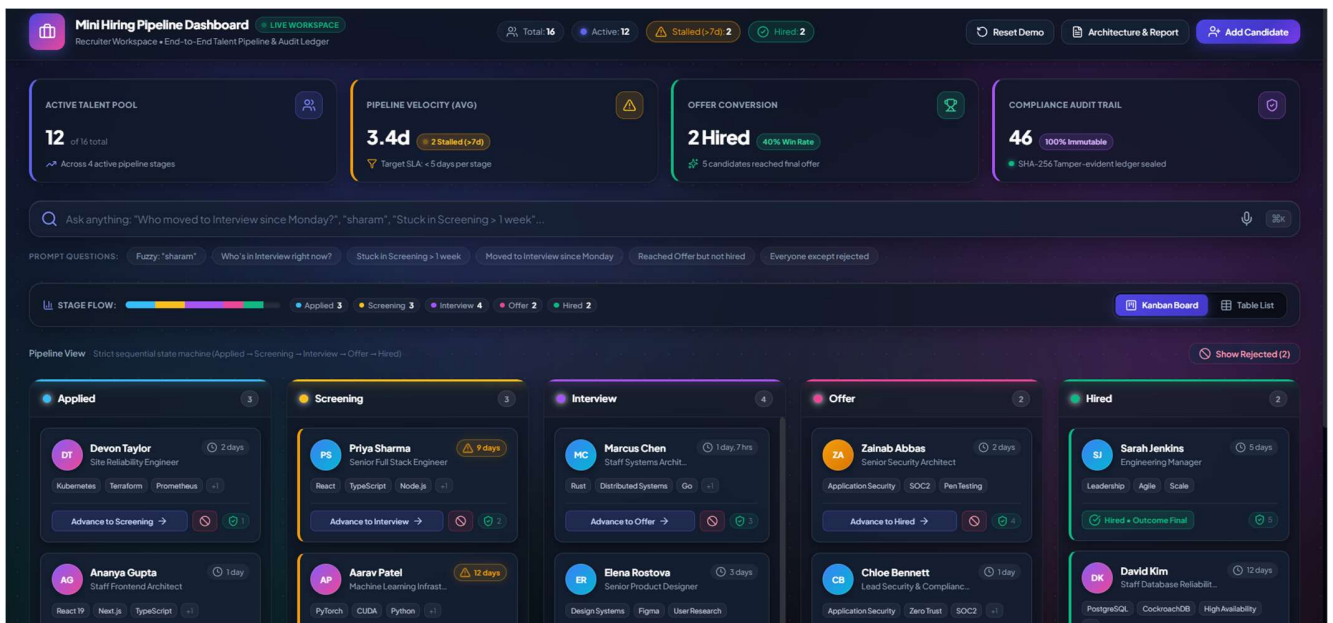
- “Who moved to Interview since Monday?” — checks the audit history for recent Interview transitions.
- “Who reached the Offer stage but did not get hired?” — checks Offer history and final outcome.
- “Everyone except rejected candidates.” — excludes rejected candidates.

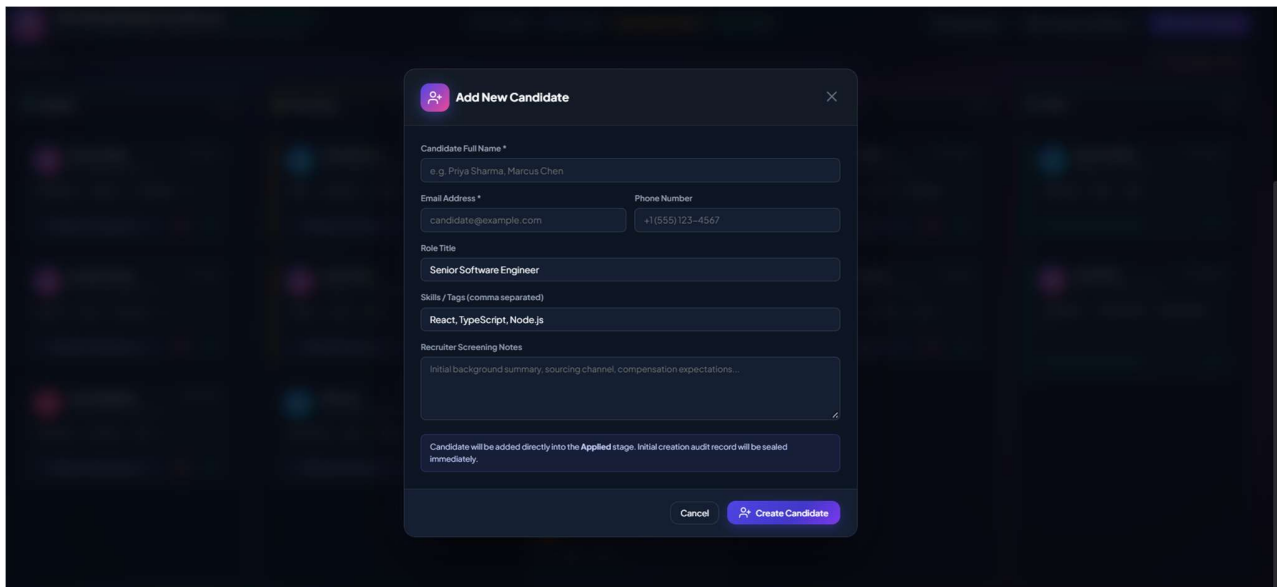
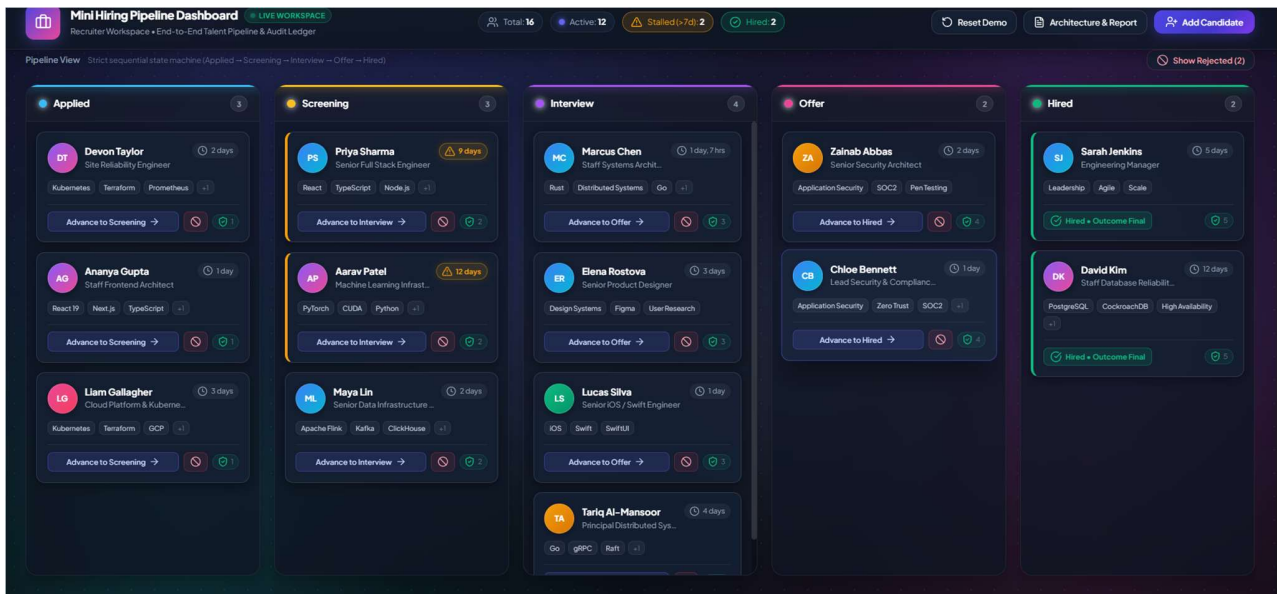
10. Search Ranking

Search results are ranked so stronger matches appear first. Name similarity, current stage, historical milestones and time-based conditions contribute to the ranking. Fuzzy matching uses Damerau-Levenshtein distance so small spelling mistakes can still produce useful results.

11. User Interface

- Dashboard with pipeline and hiring metrics.
- Stage distribution and candidate counts.
- Kanban-style candidate pipeline.
- Table/list view for candidates.
- Candidate detail drawer with history and current-stage duration.
- Add Candidate form.
- Natural-language search interface.
- Architecture/design information inside the application.
- Reset demo data functionality.





12. AI / GenAI Decision

An LLM is not required to run this application. The assignment says that AI tools are welcome; it does not require the final application to depend on an LLM.

For the final implementation, deterministic local search was preferred over sending recruiter queries to a cloud LLM. This keeps the application fast, predictable, private for candidate information, and independent of API keys, quotas and network availability.

If an LLM were added in the future, it should only convert natural-language input into a constrained, validated search intent. The application should continue to enforce all pipeline and audit rules itself.

13. Testing & Verification

The repository contains an automated verification script for the pipeline state machine, terminal states, audit behavior, duration calculations, fuzzy matching and the assignment's search scenarios.

Main commands:

```
npm install
```

```
npm test
```

```
npm run build
```

14. How to Run

Install Node.js and npm, clone the GitHub repository, install the dependencies, and start the Vite development server.

```
git clone https://github.com/SAfsehEhsani/Mini-Hiring-Pipeline-Career-Passport-.git
```

```
cd Mini-Hiring-Pipeline-Career-Passport-
```

```
npm install
```

```
npm run dev
```

Then open the local URL shown by Vite, normally `http://localhost:5173`.

15. What I Would Improve With More Time

- Add a real backend and PostgreSQL database for multi-user persistence.
- Add authentication and recruiter permissions.
- Move audit storage to a server-side append-only event store.
- Use production-grade cryptographic audit signatures.
- Add more advanced search and configurable hiring stages.
- Add automated SLA notifications.
- Add CI/CD and broader end-to-end testing.
- Optionally add a schema-constrained LLM layer for more flexible search while keeping deterministic validation.

16. Submission Checklist

- ☐ GitHub repository link included above.
- ☐ README contains setup and run instructions.
- ☐ Architecture and design decisions documented.
- ☐ AI collaboration and disagreement documented in the repository.
- ☐ Application can be demonstrated without an LLM or API key.
- ☐ Assignment search examples are supported.

☐ One-take recording can demonstrate the complete workflow.